

Daily Research report

Subject:

Implementation of a runtime for
executing computations on
volunteer machines in a
resource-limited environment

By Ngnapa Ngoule Ashley

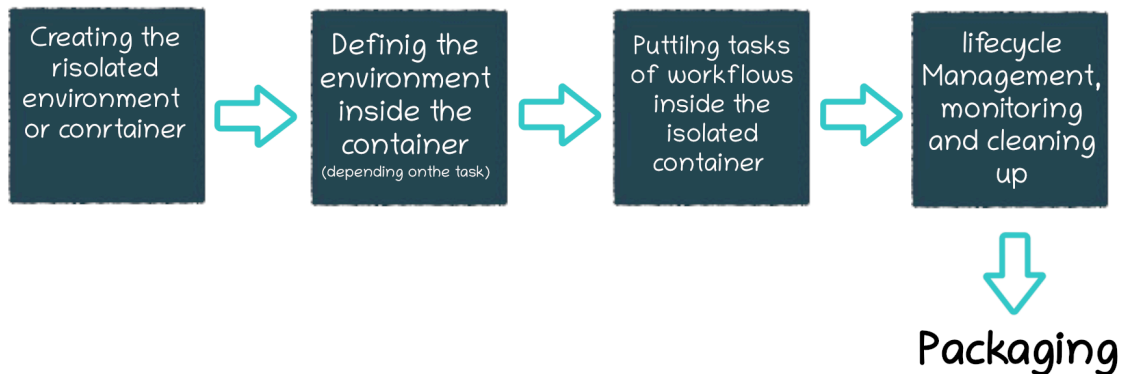
11/10/2025

Tasks achieved:

- Defining the general architecture of the final runtime to implement.
- Reading the Grok article and testing some namespaces working mechanism

General architecture of the final runtime to implement

Conceptual overview of the final runtime implementation



1. Creating the Isolated Environment or Container

In this initial step, I will establish the foundational isolation boundaries using Linux namespaces and cgroups to be able to implement the core runtime engine that can spawn these isolated environments on demand, essentially creating a lightweight container. This requires low-level programming in C or a systems language like Rust (still confused). The expected result is a sandboxed environment where tasks can run without interfering with the host system or other tasks.

2. Defining the Environment Inside the Container

Once the isolated container exists, I willt configure its internal environment to make it suitable for running actual workloads. This includes setting up the filesystem structure; mounting necessary filesystems, etc... and being able to install necessary tools inside the container to

properly run tasks. This is to create a complete, self-contained execution environment where tasks have everything they need to run successfully.

3. Putting Tasks or Workflows Inside the Isolated Container

Here we should define a mechanism to inject and execute actual work inside the container. This involves creating a task scheduler or command execution interface that can accept task definitions (commands, scripts, binaries, or workflow specifications) and spawn them as processes within the isolated namespace. We should also have a way to provide the final result of a task execution and send it to the workflow manager.

4. Lifecycle Management, Monitoring, and Cleaning Up

Here we should define some health checking mechanisms to periodically verify that tasks are still responsive and functioning correctly. Also define some graceful shutdown procedures; error handling, restarts,... that send appropriate signals (SIGTERM, then SIGKILL if necessary) to terminate, check and restart processes cleanly.

The cleanup phase is for properly unmount filesystems, destroy namespace references, c-group resources, remove temporary files and directories, to allow the volunteer to remove the system from his computer when he needs.

5. Packaging

The final deliverable step will be to transform the runtime implementation into a multiplatform distributable binary that volunteers can easily use. This involves compiling the code into a static binary and sets appropriate file permissions and provides clear usage documentation. itself.

First studying Isolation Methods

Before implementing any of these steps, I will first begin by studying the different isolation methods available in Linux, particularly focusing on understanding how each namespace type works, what isolation guarantees they provide, and what their limitations are.

Global summary of the article on namespaces and c-groups

Namespaces constitute an abstraction of the Linux kernel allowing the process to operate in an isolated way.

There are 8 types allowing us to isolate each aspect. They are

- Mount : isolate file mount points in order to design different tree structures for each namespace. There are three main files used by Mount namespaces to display information about a certain system hierarchy to the process we have:
 - ❖ Proc/pid/mount info
 - ❖ Proc/pid/mount
 - ❖ Proc/pid/mount_stats

Apart from that all the order isolated information will be different depending on the PID of the current process

Test: To test the unshare command we:

- ❖ create two different terminals in the same test environment
- ❖ create a namespace in one of those terminals
- ❖ check the PID of both process using the red link self command
- ❖ check the current Mount in each namespace
- ❖ count the number of mounts that correspond to the number of lines in the proc/self/mount file in each name space

- ❖ check the mount line the new directory in each namespace
- ❖ Testing isolation by creating a new Mount point in the new namespace display its information and check whether it is present in the host namespace

RMQ: it was not present. Cmd: Unshare -mount bash

- The UTS that isolates the hostname: the first goal of the UTS namespace isolation is to isolate the network information domain (nis) and the hostname of a new namespace. the commands used are *sethostname*, *gethostname*, *uname*, *setdomainname*, *getdomainname*.

Test: here we did the same test as previous but now considering UTS namespace

cmd: sudo nsenter -t *pid* -u hostname used to inter inside another namespace using its hostname

- the PID isolation: processes in the same number same space pid see their pid starting at 1 independently of the rest of the system (called the **init process**). This means that processes in different PID namespaces can have the same PID. This feature allows an isolated environment to be able to migrate to another host without stopping its processes.

Each process keeps its pid while running so we can resume or suspend some processes in the container without affecting the exterior environment.

cmd: Unshare -pid -mount-proc -fork bash

Tests: testing the creation of pid namespaces without -fork

Issues: what we call the **zombie building**. This appears when we try to execute commands

inside the namespace rules in it process has already been killed

Test: the test executed here was

- ❖ open two terminals. In the first create a PID namespace and in the second just conceive the reference to that namespace inside the file descriptor of the old namespace.
- ❖ create processes in the new namespace and exit to close the init process only: at this stage all the processes in the new namespace still exist but a are now orphaned
- ❖ try to inter inside the killed namespace and execute some commands

RMQ: any command refuse to execute, saying that

cmd: `sudo nsenter -pid= /proc/self/fd/3 bash`